

DEVELOPER ONBOARDING

ReactJS Developer Prerequisite & Hands-on Learning Guide

From environment setup → Git → first React project → basic React development

Prepared for: New Intern / Developer Onboarding

Prerequisites: Basic HTML, CSS, JavaScript

Goal: Be able to build and ship a basic React application independently

Contents

1. Development Environment Setup
2. Setup Visual Studio Code
3. Create Personal Git Account
4. Create Your First Personal React Repository
5. Create Git Repository
6. ReactJS Hands-on Learning Path
7. Hands-on 1 – Understand JSX
8. Hands-on 2 – Create Components
9. Hands-on 3 – Props
10. Hands-on 4 – State
11. Hands-on 5 – Form Handling
12. Hands-on 6 – Render a List
13. Hands-on 7 – useEffect
14. Hands-on 8 – Call an API
15. Hands-on 9 – React Router
16. Completion Checklist

1. Development Environment Setup

Required software

Software	Recommended
Node.js	22 LTS
npm	Comes with Node.js
Visual Studio Code	Latest stable
Git	Latest stable
Browser	Chrome / Edge
GitHub account	Personal account

Verify installation

```
node --version
npm --version
git --version
code --version
```

All commands should return a version number.

2. Setup Visual Studio Code

Required extensions

- ESLint
- Prettier – Code formatter
- GitLens

Recommended extensions

- Error Lens
- Auto Rename Tag
- Path Intellisense
- ES7+ React/Redux/React-Native Snippets

VS Code settings

Enable **Format On Save**.

Recommended `.vscode/settings.json` :

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  }
}
```

3. Create Personal Git Account

Use your own personal GitHub account rather than sharing someone else's account.

Go to GitHub and create an account. After creating your account, configure Git locally:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Check:

```
git config --global --list
```

You will then be given access to the company's Git repository through your personal GitHub account, if the company uses GitHub.

4. Create Your First Personal React Repository

Before working on the actual company project, create a small personal project to learn Git and React. For example: `react-learning`

Create the project

```
npm create vite@latest react-learning
```

Select:

- Framework: **React**
- Variant: **TypeScript**

Then:

```
cd react-learning  
npm install  
npm run dev
```

Open: `http://localhost:5173`

5. Create Git Repository

Inside the project:

```
git init
git add .
git commit -m "Initial React project"
```

Then create a new repository in your personal GitHub account: `react-learning`

Connect your local project:

```
git remote add origin https://github.com/YOUR_USERNAME/react-learning.git
```

Push:

```
git branch -M main
git push -u origin main
```

You should now be able to see the project in your personal GitHub repository.

Basic Git workflow

You should understand this workflow:

```
git pull

# Make changes.

git status
git add .
git commit -m "Add counter component"
git push
```

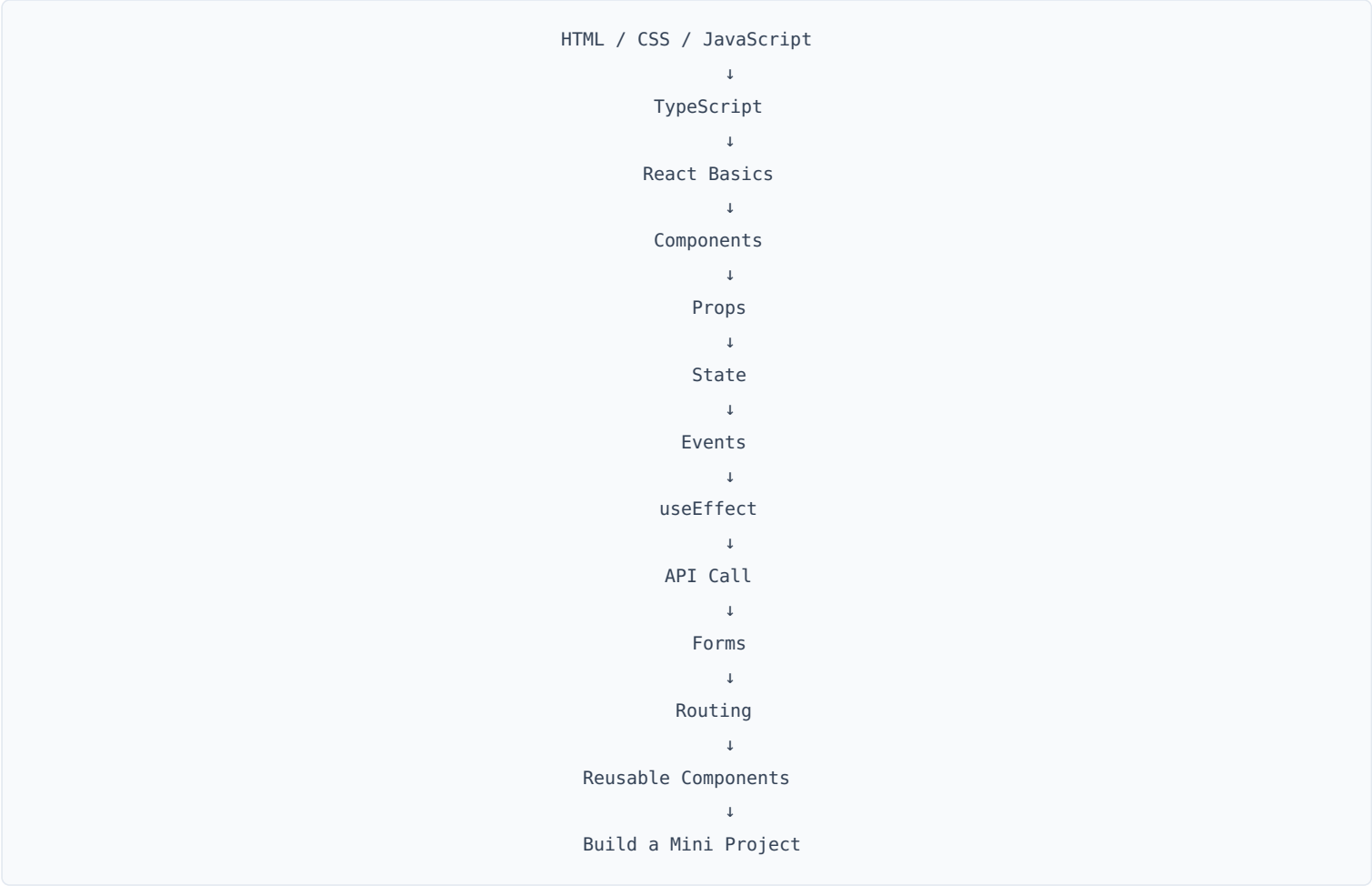
You should be comfortable with:

- `git clone`
- `git pull`
- `git status`
- `git add`
- `git commit`
- `git push`
- `git branch`
- `git checkout` / `git switch`
- Pull Request
- Merge conflict

6. ReactJS Hands-on Learning

Don't just read the React documentation — complete each hands-on exercise below for the matching concept.

Learning progression



1 Understand JSX

Modify `src/App.tsx` to create a simple profile:

```
function App() {
  const name = "John";
  const role = "Frontend Developer";

  return (
    <div>
      <h1>Hello, {name}</h1>
      <p>Role: {role}</p>
    </div>
  );
}

export default App;
```

You should understand: JSX, JavaScript expressions inside `{ }`, components, `return`, `export default`.

2 Create Components

Create:

```
src/components/Header.tsx
src/components/UserCard.tsx
```

Header

```
function Header() {
```

```
    return <h1>My React Application</h1>;
  }

export default Header;
```

UserCard

```
function UserCard() {
  return (
    <div>
      <h2>John</h2>
      <p>Frontend Developer</p>
    </div>
  );
}

export default UserCard;
```

Use them in `App.tsx` :

```
import Header from "../components/Header";
import UserCard from "../components/UserCard";

function App() {
  return (
    <>
      <Header />
      <UserCard />
    </>
  );
}

export default App;
```

You should understand component-based development.

3 Props

Modify `UserCard` :

```
interface UserCardProps {
  name: string;
  role: string;
}

function UserCard({ name, role }: UserCardProps) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

export default UserCard;
```

Then:

```
<UserCard
  name="John"
  role="Frontend Developer"
/>

<UserCard
  name="Sarah"
  role="UI/UX Designer"
/>
```

You should understand: Props = data passed from parent → child component.

4 State

Create a counter:

```
import { useState } from "react";
```

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        +1
      </button>

      <button onClick={() => setCount(count - 1)}>
        -1
      </button>
    </div>
  );
}

export default Counter;
```

Learn: `useState` , `setState` , `onClick` , re-render. This is one of the most important React concepts.

5 Form Handling

Create a simple login form:

```
import { useState } from "react";

function LoginForm() {
  const [username, setUsername] = useState("");

  const handleSubmit = (event: React.FormEvent) => {
    event.preventDefault();
    console.log("Username:", username);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={username}
        onChange={(event) => setUsername(event.target.value)}
        placeholder="Username"
      />

      <button type="submit">
        Login
      </button>
    </form>
  );
}

export default LoginForm;
```

Understand: controlled components, `onChange` , `onSubmit` , form state, event handling.

6 Render a List

Create an array:

```
const users = [
  { id: 1, name: "John" },
  { id: 2, name: "Sarah" },
  { id: 3, name: "David" }
];
```

Render it:

```
{users.map((user) => (
  <div key={user.id}>
    {user.name}
  </div>
))}
```

You should understand: `.map()` , `key` , rendering dynamic data, array/object handling.

7 useEffect

Create:

```
import { useEffect, useState } from "react";

function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    console.log("Component loaded");
  }, []);

  return (
    <div>
      <h2>Users</h2>
    </div>
  );
}

export default UserList;
```

Then explain the purpose of:

```
useEffect(() => {

}, []);
```

You should understand *when and why* side effects happen, rather than simply memorizing the syntax.

8 Call an API

Use a public test API:

```
useEffect(() => {
  fetch("https://jsonplaceholder.typicode.com/users")
    .then((response) => response.json())
    .then((data) => {
      setUsers(data);
    });
}, []);
```

Display:

```
{users.map((user) => (
  <div key={user.id}>
    {user.name}
  </div>
))}
```

Then improve it with: Loading state, Error handling, Empty result, Successful result.

This is important because you will eventually need to integrate backend APIs.

9 React Router

Install:

```
npm install react-router-dom
```

Create pages:

```
src/pages/
├── Home.tsx
├── Users.tsx
└── Settings.tsx
```

Learn basic routing:

```
/
/users
/settings
```

You should understand: Routes, Navigation, Route parameters, Nested routes, Protected routes.

Completion Checklist

Before starting on the actual project, you should be able to demonstrate the following:

Environment

- ☐ Install Node.js
- ☐ Install VS Code
- ☐ Install Git
- ☐ Configure VS Code
- ☐ Run React application locally

Git

- ☐ Create personal GitHub account
- ☐ Create repository
- ☐ Clone repository
- ☐ Create branch
- ☐ Commit code
- ☐ Push code
- ☐ Create Pull Request
- ☐ Resolve merge conflict

React

- ☐ Understand JSX
- ☐ Create components
- ☐ Pass props
- ☐ Use useState
- ☐ Handle events
- ☐ Handle forms
- ☐ Render lists
- ☐ Use useEffect
- ☐ Call REST API
- ☐ Handle loading/error states
- ☐ Use React Router
- ☐ Understand basic TypeScript
- ☐ Build reusable components

Final Submission

- ☐ Push project to personal GitHub
- ☐ Create Pull Request
- ☐ Reviewer checks code quality
- ☐ `npm run lint` passes
- ☐ `npm run build` passes

Follow this path — from installing the tools to actually proving you can build a basic React application — before touching the main project.